

Un linguaggio per domarli tutti: applicazioni full stack con librerie native in Kotlin Multiplatform

Sohaib Ouakani

Corso di Laurea in Ingegneria e Scienze informatiche

Relatore: Prof. Danilo Pianini

Correlatori: Dott. Angela Cortecchia, Ing. Valter Venusti

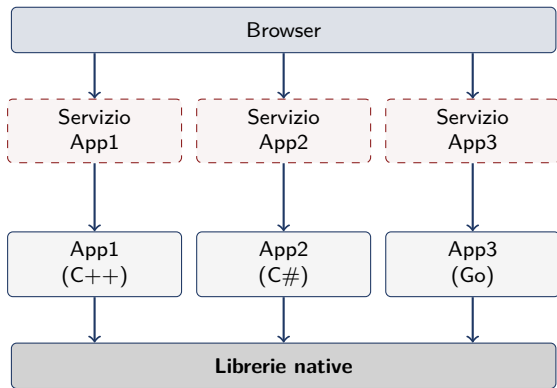
A.A. 2025/2026

Contesto: stack software eterogeneo

- Stack industriali spesso eterogenei
- Ogni strumento è scritto in linguaggi diversi
- Librerie di standard tecnici scritte in linguaggi nativi

Problema

Duplicazione di codice, complessità, manutenzione difficile



Applicativi diversi, ma stesse librerie

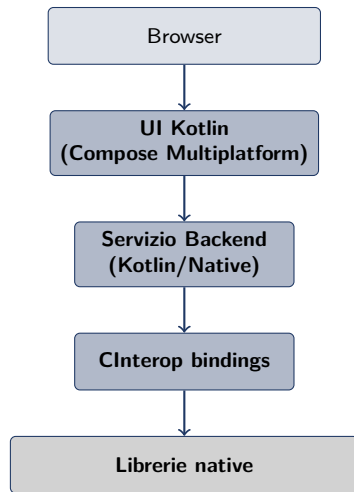
- Node.js: molto popolare, ma interazione con il nativo complessa
- Linguaggi fullstack popolari richiedono spesso codice in altri linguaggi

Limite principale

Serve codice legante in C per interagire con librerie native

Kotlin Multiplatform + CInterop

- Un solo linguaggio per tutto lo stack: Kotlin
- Interfaccia diretta con C tramite CInterop
- Nessun layer di astrazione intermedio



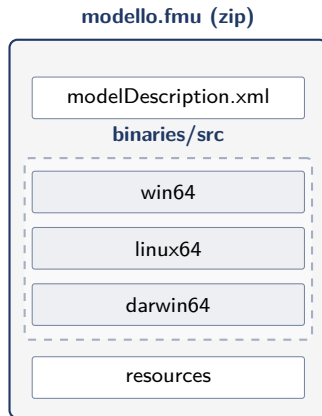
Stesso linguaggio per UI e servizio

Standard FMI e formato FMU

- FMI: standard per lo scambio di modelli di simulazione, con interfaccia C
- FMU: archivio zip con modello e metadati, secondo lo standard FMI

Perché sono rilevanti

Caso di studio per validare l'approccio Kotlin fullstack in ambito industriale



Struttura standard di un file FMU secondo FMI 2.0.

Architettura del sistema



Figura: Architettura generale del sistema.

Linguaggi

- Frontend: **Kotlin/JS**
- Backend: **Kotlin/Native**
- Logica di dominio (fmu-kt): **Kotlin/Native**
- Stessa sintassi e semantica
- Librerie, backend e runtime diversi

Dettaglio dell'architettura

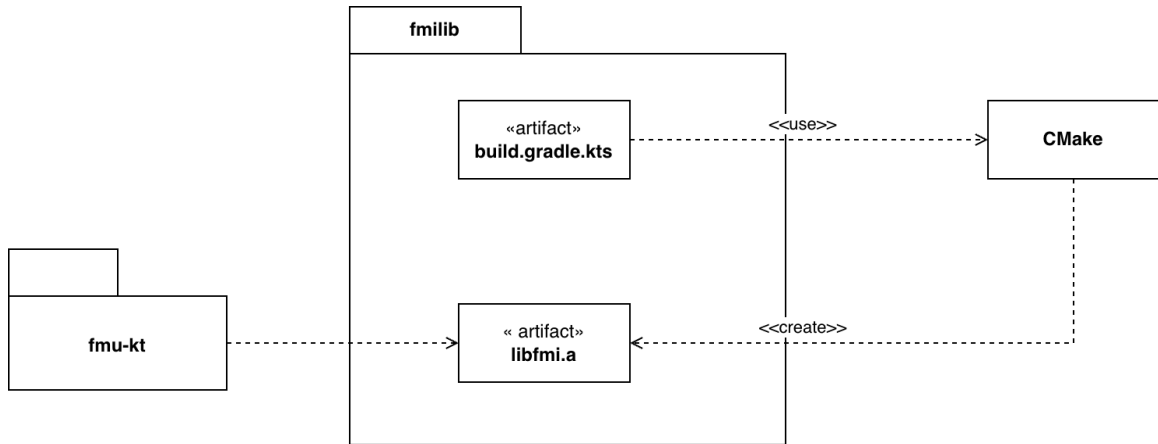
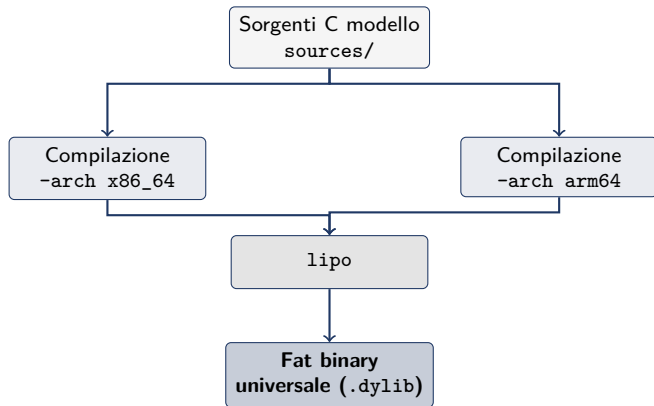


Figura: Come avviene l'integrazione con il codice nativo.

Supporto sperimentale a sistemi Apple Silicon

- FMU spesso solo in x86_64, incompatibili con Apple Silicon
- Rosetta 2 non utilizzabile (caricamento dinamico via dlopen)



- Test unitari dei moduli
- Test di integrazione su FMU pubblici (submodule git + Kotest)
- CI su GitHub Actions: Windows, Linux, macOS

Conclusioni

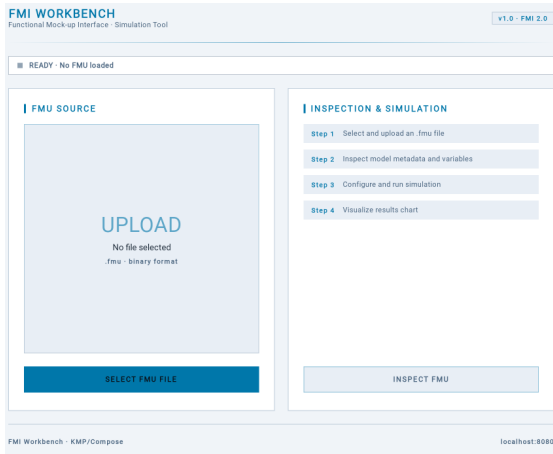


Figura: Schermata di upload.

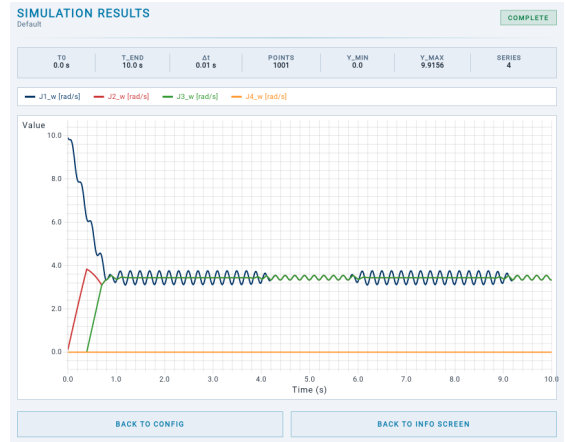


Figura: Grafico di una simulazione.

Obiettivi raggiunti

- Caricamento FMU
- Visualizzazione metadati e variabili
- Configurazione simulazione
- Visualizzazione risultati simulazione

Limitazioni

- Solo standard FMI 2.0
- Ricompilazione macOS best effort
- Kotlin/Native: stdlib limitata

Sviluppi futuri

- Supporto ad altre versioni FMI
- Esportazione risultati simulazione
- Gestione più modelli FMU in contemporanea